

LOGIC BUILDING MASTERY GUIDE

■ Java Edition ■

A Step-by-Step System to Solve Any Problem in Java

7-Step Framework • 10 Core Patterns • 10 Solved Java Problems



Think Like a Programmer



Pure Java Code



Step-by-Step Framework



10 Solved Problems

SECTION 1 | What is Logic Building?

What is Logic Building?

Logic building is the ability to break a complex problem into small, clear, executable steps and then translate those steps into code. In Java this is especially important because Java is strongly typed and verbose — you need to plan before you type, or you'll be lost in syntax.

Why Java Beginners Struggle

- Java requires declaring types — beginners confuse thinking about logic with thinking about types.
- The `main()` method structure feels rigid; they don't know where to start.
- They jump to complex OOP before mastering basic control flow.
- Error messages are verbose and confusing, causing panic instead of reasoning.

The Good News

The logic is identical in every language. Once you master the thinking framework, Java syntax becomes easy to fill in. This guide teaches you to THINK first, code second.

Java Program Structure — Know This First

Basic Java Structure

```
// Every Java program starts here
public class Main {
    public static void main(String[] args) {
        // Your logic goes here
        System.out.println("Hello, Logic!");
    }
}
```

The 3 Pillars of Logical Thinking

PILLAR	MEANING	QUESTION TO ASK
■ Understand	Grasp what the problem truly needs	What is the input? What is the output?
➤■ Decompose	Break big problems into small steps	What are the sub-tasks?
■ Pattern	Reuse known solutions	Have I seen something like this before?

SECTION 2 | The 7-Step Problem-Solving Framework

Apply These 7 Steps to Every Problem

These 7 steps work for any Java problem — whether it's a coding interview, a university assignment, or a real project feature.

1 READ & UNDERSTAND

Read twice. Identify: input type (int, String, array?), expected output, constraints (array size, negative numbers, null?).

2 WORK EXAMPLES BY HAND

Solve 2-3 small examples manually. Trace through the logic with real values before writing any code.

3 IDENTIFY THE PATTERN

Is this a loop? A condition? A search? A sort? Map it to a known pattern. Most problems repeat 10-15 patterns.

4 WRITE PSEUDOCODE

Write plain English steps. e.g. 'For each element in array, if element is even, add to sum.' Bridge thinking and Java.

5 TRANSLATE TO JAVA

Convert pseudocode to Java line by line. Think: what type? what method? Don't worry about perfect code — get it working.

6

TEST WITH EDGE CASES

Test: empty array, single element, null, very large values, negative numbers, duplicates. Java will throw exceptions if you miss these.

7

REFLECT & OPTIMIZE

Can you use a HashMap instead of nested loops? Can you reduce from $O(n^2)$ to $O(n)$? What pattern did you use? Write it down.

SECTION 3 | 10 Core Logic Patterns in Java

90% of all programming problems reduce to one of these 10 patterns. Recognizing the pattern is the biggest leap toward the solution.

#	PATTERN	JAVA TOOLS	KEY CLUE WORDS
1	Counting / Accumulator	<code>int sum=0; for loop</code>	"total", "count", "sum"
2	Linear Search	<code>for + if condition</code>	"find", "exists", "contains"
3	Two Pointers	<code>int left=0, right=n-1</code>	"pair", "palindrome", "sorted"
4	Sliding Window	<code>int start, end; while loop</code>	"consecutive", "subarray"
5	Frequency Map	<code>HashMap<Character,Integer></code>	"duplicate", "most frequent"
6	Sorting First	<code>Arrays.sort(arr)</code>	"kth smallest", "median"
7	Recursion	method calls itself	"tree", "factorial", "subsets"
8	Greedy	pick local best each step	"minimum steps", "max profit"
9	Binary Search	<code>Arrays.binarySearch</code> or manual	"sorted", " $O(\log n)$ "
10	Stack / Queue	<code>Stack<></code> , <code>LinkedList<></code> as Queue	"brackets", "BFS", "order"

SECTION 4 | 10 Solved Problems with Full Java Walkthrough

Each problem is solved using ALL 7 steps. Read the hand-trace and pseudocode BEFORE the Java code — that is the most important habit you can build.

Problem 1 | Sum of All Even Numbers in an Array

Pattern: Counting / Accumulator

Input: `int[] nums = {1, 2, 3, 4, 5, 6}`

Output: 12 (2+4+6)

Step 2 — Work by Hand

Walk through each number. If it divides by 2 with no remainder, add to a running total. $2+4+6 = 12$.

Step 4 — Pseudocode

Pseudocode (language-agnostic)

```
SET total = 0
FOR each number in array:
    IF number % 2 == 0:
        ADD number to total
RETURN total
```

Step 5 — Java Code

Java

```
public class Main {
    public static int sumEven(int[] nums) {
        int total = 0;
        for (int num : nums) {
            if (num % 2 == 0) { // even check
                total += num;
            }
        }
        return total;
    }
    public static void main(String[] args) {
        int[] nums = {1, 2, 3, 4, 5, 6};
        System.out.println(sumEven(nums)); // 12
    }
}
```

Problem 2 | Check if a String is a Palindrome

Pattern: Two Pointers

Input: `"racecar"`

Output: true

Step 2 — Work by Hand

Compare $r==r$, $a==a$, $c==c$ — all match from both ends inward. It is a palindrome.

Step 4 — Pseudocode

Pseudocode (language-agnostic)

```
SET left = 0, right = s.length() - 1
WHILE left < right:
    IF s.charAt(left) != s.charAt(right): RETURN false
    INCREMENT left, DECREMENT right
RETURN true
```

Step 5 — Java Code

Java

```
public class Main {
    public static boolean isPalindrome(String s) {
        int left = 0, right = s.length() - 1;
        while (left < right) {
            if (s.charAt(left) != s.charAt(right)) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isPalindrome("racecar")); // true
        System.out.println(isPalindrome("hello"));  // false
    }
}
```

Problem 3 | Find the Largest Number in an Array

Pattern: Linear Search / Accumulator

Input: `int[] arr = {3, 7, 1, 9, 4}`

Output: 9

Step 2 — Work by Hand

Assume first element is max (3). Compare $7 > 3 \rightarrow$ new max=7. Compare $9 > 7 \rightarrow$ new max=9. Done.

Step 4 — Pseudocode

Pseudocode (language-agnostic)

```
SET max = arr[0]
FOR each number in arr:
    IF number > max: SET max = number
RETURN max
```

Step 5 — Java Code

Java

```
public class Main {
    public static int findMax(int[] arr) {
        int max = arr[0];
        for (int num : arr) {
            if (num > max) {
                max = num;
            }
        }
        return max;
    }
    public static void main(String[] args) {
        int[] arr = {3, 7, 1, 9, 4};
        System.out.println(findMax(arr)); // 9
    }
}
```

Problem 4 | Count Frequency of Each Character

Pattern: Frequency Map (HashMap)

Input: "hello"

Output: {h=1, e=1, l=2, o=1}

Step 2 — Work by Hand

Walk each letter: h->1, e->1, l->1, l becomes 2, o->1. Use a map to store counts.

Step 4 — Pseudocode

Pseudocode (language-agnostic)

```
CREATE empty HashMap<Character, Integer> freq
FOR each char in string:
    IF char in freq: freq[char] += 1
    ELSE: freq[char] = 1
RETURN freq
```

Step 5 — Java Code

Java

```
import java.util.HashMap;
public class Main {
    public static HashMap<Character,Integer> charFreq(String s) {
        HashMap<Character, Integer> freq = new HashMap<>();
        for (char ch : s.toCharArray()) {
            freq.put(ch, freq.getOrDefault(ch, 0) + 1);
        }
        return freq;
    }
    public static void main(String[] args) {
        System.out.println(charFreq("hello"));
        // {h=1, e=1, l=2, o=1}
    }
}
```

Problem 5 | Fibonacci Sequence (First N Terms)

Pattern: Accumulator + Pattern Recognition

Input: `n = 7`

Output: `[0, 1, 1, 2, 3, 5, 8]`

Step 2 — Work by Hand

Start: 0, 1. Each next = sum of previous two: 0+1=1, 1+1=2, 1+2=3, 2+3=5, 3+5=8.

Step 4 — Pseudocode

Pseudocode (language-agnostic)

```
CREATE int[] seq of size n
seq[0] = 0, seq[1] = 1
FOR i from 2 to n-1:
    seq[i] = seq[i-1] + seq[i-2]
RETURN seq
```

Step 5 — Java Code

Java

```
import java.util.Arrays;
public class Main {
    public static int[] fibonacci(int n) {
        int[] seq = new int[n];
        if (n >= 1) seq[0] = 0;
        if (n >= 2) seq[1] = 1;
        for (int i = 2; i < n; i++) {
            seq[i] = seq[i-1] + seq[i-2];
        }
        return seq;
    }
    public static void main(String[] args) {
        System.out.println(Arrays.toString(fibonacci(7)));
        // [0, 1, 1, 2, 3, 5, 8]
    }
}
```

Problem 6 | Two Sum — Find Indices of Pair

Pattern: Frequency Map
(HashMap for O(n))

Input: `int[] nums={2,7,11,15}, target=9`

Output: `[0, 1]`

Step 2 — Work by Hand

For each number, check if (target - number) was already seen. 9-2=7; not seen yet. 9-7=2; YES, seen at index 0!

Step 4 — Pseudocode

Pseudocode (language-agnostic)

```
CREATE HashMap<Integer,Integer> seen (value -> index)
FOR i = 0 to nums.length-1:
    SET complement = target - nums[i]
    IF complement in seen: RETURN [seen[complement], i]
    PUT nums[i] -> i in seen
RETURN empty (no solution)
```

Step 5 — Java Code

Java

```
import java.util.HashMap;
import java.util.Arrays;
public class Main {
    public static int[] twoSum(int[] nums, int target) {
        HashMap<Integer, Integer> seen = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (seen.containsKey(complement)) {
                return new int[]{seen.get(complement), i};
            }
            seen.put(nums[i], i);
        }
        return new int[]{};
    }
    public static void main(String[] args) {
        int[] result = twoSum(new int[]{2,7,11,15}, 9);
        System.out.println(Arrays.toString(result)); // [0, 1]
    }
}
```

Problem 7 | Reverse an Array Without Built-ins

Pattern: Two Pointers

Input: `int[] arr = {1, 2, 3, 4, 5}`

Output: `[5, 4, 3, 2, 1]`

Step 2 — Work by Hand

Swap first and last (1<->5), then second and second-to-last (2<->4). Middle (3) stays.

Step 4 — Pseudocode

Pseudocode (language-agnostic)

```
SET left = 0, right = arr.length - 1
WHILE left < right:
    SWAP arr[left] and arr[right]
    INCREMENT left, DECREMENT right
```

Step 5 — Java Code

Java

```
import java.util.Arrays;
public class Main {
    public static void reverseArray(int[] arr) {
        int left = 0, right = arr.length - 1;
        while (left < right) {
            int temp = arr[left]; // swap
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
    }
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        reverseArray(arr);
        System.out.println(Arrays.toString(arr)); // [5,4,3,2,1]
    }
}
```

Problem 8 | Check Balanced Parentheses

Pattern: Stack

Input: `"(())"`

Output: `true`

Step 2 — Work by Hand

Push opening brackets. On closing bracket, pop and check if it matches. If stack empty at end -> balanced.

Step 4 — Pseudocode

Pseudocode (language-agnostic)

```
CREATE Stack<Character> stack
FOR each char in string:
    IF char is '(' '{' '[': PUSH to stack
    IF char is ')' '}' ']':
        IF stack empty OR top doesn't match: RETURN false
        POP from stack
RETURN stack.isEmpty()
```

Step 5 — Java Code

Java

```
import java.util.Stack;
public class Main {
    public static boolean isBalanced(String s) {
        Stack<Character> stack = new Stack<>();
        for (char ch : s.toCharArray()) {
            if (ch=='(' || ch=='{' || ch=='[') {
                stack.push(ch);
            } else if (ch==')' || ch=='}' || ch==']') {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if ((ch==')' && top!='(') ||
                    (ch=='}' && top!='{') ||
                    (ch==']' && top!='[')) {
                    return false;
                }
            }
        }
        return stack.isEmpty();
    }
    public static void main(String[] args) {
        System.out.println(isBalanced("({[]})")); // true
        System.out.println(isBalanced("[]"));      // false
    }
}
```

Problem 9 | Binary Search in Sorted Array

Pattern: Binary Search (Divide and Conquer)

Input: `int[] arr={1,3,5,7,9,11}, target=7`

Output: 3 (index)

Step 2 — Work by Hand

Mid=index 2 (val 5). $7 > 5$, search right. Mid=index 4 (val 9). $7 < 9$, search left. Mid=index 3 (val 7). Found!

Step 4 — Pseudocode

Pseudocode (language-agnostic)

```
SET low=0, high=arr.length-1
WHILE low <= high:
    SET mid = (low+high) / 2
    IF arr[mid] == target: RETURN mid
    IF arr[mid] < target: low = mid + 1
    ELSE: high = mid - 1
RETURN -1 (not found)
```

Step 5 — Java Code

Java

```
public class Main {
    public static int binarySearch(int[] arr, int target) {
        int low = 0, high = arr.length - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (arr[mid] == target) {
                return mid;
            } else if (arr[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] arr = {1,3,5,7,9,11};
        System.out.println(binarySearch(arr, 7)); // 3
        System.out.println(binarySearch(arr, 4)); // -1
    }
}
```

Problem 10 | FizzBuzz

Pattern: Condition Chains

Input: `n = 15`

Output: 1, 2, Fizz, 4, Buzz, ..., FizzBuzz

Step 2 — Work by Hand

Check divisibility by both 3 and 5 FIRST (most specific), then 3 only, then 5 only, then number.

Step 4 — Pseudocode

Pseudocode (language-agnostic)

```
FOR i from 1 to n:
    IF i % 3==0 AND i % 5==0: PRINT FizzBuzz
    ELSE IF i % 3==0: PRINT Fizz
    ELSE IF i % 5==0: PRINT Buzz
    ELSE: PRINT i
```

Step 5 — Java Code

Java

```
public class Main {
    public static void fizzBuzz(int n) {
        for (int i = 1; i <= n; i++) {
            if (i % 3 == 0 && i % 5 == 0) {
                System.out.print("FizzBuzz ");
            } else if (i % 3 == 0) {
                System.out.print("Fizz ");
            } else if (i % 5 == 0) {
                System.out.print("Buzz ");
            } else {
                System.out.print(i + " ");
            }
        }
    }

    public static void main(String[] args) {
        fizzBuzz(15);
        // 1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
    }
}
```

SECTION 5 | Java-Specific Tips & 30-Day Practice Plan

Java Tips Every Logic Builder Needs

TIP	DETAIL
Use <code>int[]</code> for fixed arrays	<code>int[] arr = new int[5];</code> — index from 0 to 4.
Use <code>ArrayList</code> for dynamic	<code>ArrayList<Integer> list = new ArrayList<>();</code> — add/remove freely.
Use <code>HashMap</code> for key-value	<code>HashMap<String,Integer> map = new HashMap<>();</code> — $O(1)$ lookup.
Use <code>Stack<></code> for LIFO	<code>Stack<Character> s = new Stack<>(); s.push(); s.pop();</code>
Enhanced for-loop	<code>for (int x : arr)</code> — cleaner than index loop when index not needed.
<code>getOrDefault</code> for maps	<code>map.getOrDefault(key, 0)</code> — avoids <code>NullPointerException</code> .
<code>Arrays.toString()</code>	<code>System.out.println(Arrays.toString(arr))</code> — prints arrays readable.
String to char array	<code>s.toCharArray()</code> — iterate characters easily.

Your 30-Day Java Logic Building Plan

WEEK	FOCUS	DAILY PRACTICE
Week 1 Days 1-7	Java Basics + Loops	Sums, counts, max/min. Print patterns. Write pseudocode first, then Java.
Week 2 Days 8-14	Strings + Arrays	Palindromes, reversal, frequency maps. 2 problems/day. Trace before coding.
Week 3 Days 15-21	HashMap + Sorting	Two Sum, duplicates, <code>Arrays.sort()</code> . 2-3 problems/day. Time your solution.
Week 4 Days 22-30	Stack + Binary Search + OOP	Balanced brackets, search, recursion. 3 problems/day + refactor 1 old solution.

Golden Rules for Java Logic Building

Never: Write a single line of Java before solving 2 examples by hand.

Always: Write pseudocode before touching the keyboard.

Always: Declare your variable types intentionally — it forces clearer thinking.

Always: Test with: null, empty array, single element, negative values.

Repeat: After solving: name the pattern. Write it in a notebook.

Track: Keep a problem log. Patterns you forget reappear in interviews.

Remember This Always

Every Java expert was once stuck on a HelloWorld. The difference is they never stopped solving one more problem.

You now have the Framework • The Patterns • 10 Solved Java Problems Pick one problem. Apply the 7 steps. Write the code. Repeat.
